Phase 13 — DS Style 30 Page Pack

Schedule + Procurement module documentation from uploaded implementation transcript

DS LOCKED PURPOSE: This document compresses Phase 13 into exactly thirty structured pages. It follows the user style: step-by-step, senior-engineering reasoning, colored DSRC blocks, concise diagrams, tables, definitions, and light code only where it explains architecture.

SCOPE BASIS: The source transcript shows the Phase 13 build moving from codebase introspection into Schedule and Procurement migrations, backend modules, permissions, and initial frontend Schedule UI work. This document therefore documents the implemented/visible transcript accurately and marks verification items as planned when the transcript does not show a final smoke result.

Module	Result in Phase 13	Senior meaning
Schedule	PM + Calibration schedule foundation	Planning layer for technical maintenance/calibration work.
Procurement	PO + spare-part inventory foundation	Commercial/inventory layer linked to equipment and vendors.
RBAC	Dedicated schedule/procurement permissions	Removes borrowed gates and makes navigation semantically correct.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 01 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Page Map — 30 Pages Only

Page-by-page documentation plan

READING DOCTRINE: Each page is self-contained. Page 1 defines the seal, Page 2 maps the content, Pages 3-8 capture architecture/database/RBAC, Pages 9-18 capture backend logic, Pages 19-24 capture frontend logic, and Pages 25-30 capture verification, risks, and locked handover.

Pages	Coverage	Reason
01-02	Document identity and map	Locks the reader into the exact thirty-page flow.
03-08	Context, DB, permissions	Explains why new tables and new permission gates were needed.
09-18	Backend modules	Documents validators, repositories, services, routes, ICS, CSV and transaction logic.
19-24	Frontend modules	Documents API wrappers, hooks, Schedule UI, Procurement UI intent and token behavior.
25-30	Quality, risks, seal	Makes the phase auditable and ready for continuation.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 02 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Step 0 — Codebase Introspection

Why the build starts with read-only discovery

DEFINITION: Introspection means reading the existing folder structure, modules, migrations, middleware, FE pages, and permission maps before changing code. This prevents accidental redesign and preserves the sealed doctrine from Phases 6-12.

DS SENIOR CALL: Phase 13 did not begin by inventing a fresh architecture. It inspected BE/src/modules, DATABASE/phase3/migrations, FE/src/pages, FE/src/lib/api, hooks, layout, notifications, tokens, and existing repo patterns first.

Area inspected	Why it mattered	Outcome
Backend modules	To follow validator -> repo -> service -> controller -> routes	Schedule and Procurement match the established module shape.
Migrations	To choose non-conflicting IDs	500, 501, 510 used for Phase 13.
Frontend pages/hooks	To reuse DataTable, Layout, auth and token patterns	FE is consistent with earlier phases.
Notifications/tokens	To respect Phase 12 behavior	Mutations can participate in token feedback.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 03 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Phase 13 Position in CMCMIS

Why Schedule + Procurement comes after Reports, PDF and Notifications

SYSTEM POSITION: By the time Phase 13 starts, the system already has Job Requests, Job Cards, Equipment, Dashboard, Inquiry, Reports, PDF downloads, Analytics, and Notifications. Schedule + Procurement are therefore not isolated pages; they extend the operational backbone.

RELATIONSHIP THINKING: Schedule plans future technical work. Procurement supplies material and spare support. Together they close the gap between reactive job execution and proactive engineering management.

DIAGRAM: Architecture or flow representation below.

```
Existing sealed core
  Equipment -> Job Request -> Job Card -> Report/PDF
        |             |             |
        |             |             +-> Notifications + Tokens
        |             |
        +-> Phase 13 Schedule
        +-> Phase 13 Procurement
```

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 04 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Database Decision — Additive Tables Only

Legacy-safe design for a live CMCMIS database

PRINCIPLE: No existing legacy table is dropped or redesigned. Phase 13 adds only the tables required to represent new operational concepts that the legacy schema does not model cleanly.

WHY NEW TABLES: cmms_eqip_mst stores equipment due fields and procurement fragments, but it cannot represent assignable schedule lifecycle rows, multi-line purchase orders, or standalone spare inventory. New tables are justified because they represent new domain entities.

Table	Type	Reason
schedules	New master	PM/Calibration plan row with date, status, engineer and recurrence.
schedule_status_history	New child/history	Append-only schedule transitions.
spare_parts	New master	Inventory row with stock, vendor, min stock and cost.
purchase_orders	New master	PO header with vendor, status, total and warranty.
purchase_order_items	New child	Line items with quantity, unit cost, line total.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 05 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Migration 500 — Schedule Tables

schedules + schedule_status_history

DSRC DECISION: The Schedule module uses two tables: schedules for the current operational state and schedule_status_history for transition audit. The design is event-aware but not over-engineered.

SOFT FK DOCTRINE: equipment_id is stored as a composite string like EQM_TYPE-EQM_ID. The service validates it against cmms_eqip_mst instead of adding a hard FK to legacy production tables.

Column group	Important columns	Purpose
Identity	id, schedule_code	Stable internal id plus human-readable code.
Classification	schedule_type, priority, recurrence	PM vs calibration and planning intent.
Linkage	equipment_id, equipment_label, assigned_engineer_employee_id	Connects schedule to equipment and staff.
Lifecycle	scheduled_date, status	Calendar date and operational state.
Audit	created_by_employee_id, updated_by_employee_id, timestamps	Traceability.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 06 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Migration 501 — Procurement Tables

spare_parts + purchase_orders + purchase_order_items

DSRC DECISION: Procurement is represented with an inventory table and PO header/line-item tables. This allows the system to store commercial action without forcing it into equipment asset columns.

TOTALS DOCTRINE: purchase_orders.total_cost and purchase_order_items.line_total are server-computed. The frontend never becomes the source of truth for money totals.

Entity	Main purpose	Senior notes
spare_parts	Inventory and reorder visibility	Supports stock_qty <= min_stock detection.
purchase_orders	Vendor-level commercial header	Stores status, PO date, warranty and total.
purchase_order_items	Actual PO item rows	Hard FK to PO header; soft link to spare part.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 07 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Migration 510 — RBAC Permission Matrix

Replacing borrowed gates with true Phase 13 permissions

SECURITY FIX: Before Phase 13, sidebar visibility for Schedule/Procurement could be temporarily borrowed from equipment permissions. Migration 510 gives the modules their own permission language.

VIEW-ONLY RULE: View-Only can read the new modules but cannot export, create, update, order, delete, or mutate any Phase 13 data.

Permission family	SA/LIC	Lab Engineer	Normal User	View Only
schedule:*	All schedule permissions	read/update/export	read-list only	read-list only
procurement:*	All procurement permissions	read/order/export	read-list only	read-list only
write access	Yes	Limited	No	No
export access	Yes	Yes where granted	No	No

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 08 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Schedule Backend Architecture

Validator -> State Machine -> Repo -> Service -> Controller -> Routes

MODULE SHAPE: The Schedule backend follows the sealed production module shape. Validators sanitize shape, state machine owns transitions, repo owns SQL, service owns transaction logic, controller owns HTTP shims, and routes own RBAC gates.

DIAGRAM: Architecture or flow representation below.

```
HTTP request
  -> schedule.routes.js      // authenticate + authorize + validate
  -> schedule.controller.js  // request/response only
  -> schedule.service.js     // transaction + business rules
  -> schedule.repo.js        // SQL only
  -> MySQL tables            // schedules + history + audit_log
```

File	Responsibility	What to review
schedule.validators.js	Input contract	Enums, dates, pagination, strict payloads.
schedule.stateMachine.js	Lifecycle rules	Allowed transitions and DUE derivation.
schedule.repo.js	Database access	Soft-FK checks, list, insert, update, audit.
schedule.service.js	Business flow	Transactions, ICS, lazy DUE logic.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 09 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Schedule Validators

Input perimeter and predictable API contracts

WHY VALIDATORS MATTER: Validators stop malformed or unknown fields before they reach SQL. This is especially important for an engineering system because small field mistakes can create operational ambiguity.

STRICT PAYLOAD DOCTRINE: All schedule schemas use strict validation. Extra fields are rejected rather than silently ignored.

Schema	Route usage	Key rules
listQuerySchema	GET /schedules	type/status/date range/engineer/q/page/page_size.
createSchema	POST /schedules	schedule_type, equipment_id, date, priority, engineer, recurrence.
editSchema	PATCH /schedules/:id	Partial update with at least one field.
statusTransitionSchema	POST /schedules/:id/status	Target status; reason required for CANCELLED.
icsExportQuerySchema	GET /schedules/export.ics	Filter-only bulk calendar export.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 10 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Schedule State Machine

Planned, Scheduled, Due, Completed, Cancelled

LIFECYCLE TRUTH: The state machine prevents direct illegal movement. Completed and Cancelled are terminal states. DUE can be persisted, but it is also derivable from scheduled_date.

DUE DERIVATION: If a schedule is still PLANNED or SCHEDULED and scheduled_date <= today, effective status becomes DUE. This keeps the UI honest even before a manual transition happens.

DIAGRAM: Architecture or flow representation below.

```
PLANNED -> SCHEDULED -> DUE -> COMPLETED
    |          |          |
    +----------+----------+-> CANCELLED

Terminal states: COMPLETED, CANCELLED
Derived state: scheduled_date passed => DUE
```

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 11 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Schedule Repository Logic

SQL ownership and canonical field mapping

REPO ALIASING: schedule.repo.js is the only Schedule file that owns SQL. Other layers consume canonical field names, not raw table concerns.

CODE GENERATION: nextScheduleCode builds CAL-YYYY-MM-NN or PM-YYYY-Qn-NN using a FOR UPDATE-locked count query. This protects display code generation against race conditions.

Repository function	Purpose	Senior risk controlled
findEquipmentByCompositeId	Validate soft equipment reference	Prevents orphan schedule rows.
nextScheduleCode	Generate display id	Avoids duplicate human codes.
listSchedules	Filtered list/calendar data	Single query path for UI modes.
appendStatusHistory	Write transition row	Timeline integrity.
writeAuditLog	Central audit row	Traceable operational actions.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 12 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Schedule Service Logic

Transaction boundary and business decisions

TRANSACTION RULE: Every write opens a transaction. Schedule row, schedule_status_history row, audit_log row, and notification emission must either commit together or rollback together.

TERMINAL GUARD: Completed and Cancelled schedules cannot be edited. Rework must become a new schedule row, keeping the history clear.

Service method	Main flow	Output
createSchedule	Validate equipment/engineer -> generate code -> insert -> history -> audit	id, schedule_code, status.
editSchedule	Load existing -> validate changed refs -> update -> audit	fresh detail.
transitionSchedule	derive DUE -> state machine -> update -> history -> audit	fresh detail.
cancelSchedule	Logical cancel through transition path	fresh detail.
getIcsForFilter	List schedules -> build VCALENDAR	.ics body.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 13 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

ICS Export Logic

Calendar integration without dependency bloat

WHY ICS EXISTS: Schedule data should leave CMCMIS safely into Outlook, Google Calendar, Apple Calendar or local calendar tools. The system emits RFC-5545 compatible VCALENDAR/VEVENT text.

SENIOR DESIGN: The serializer is intentionally small: UID, DTSTAMP, DTSTART/DTEND, SUMMARY, DESCRIPTION and CATEGORIES. This avoids adding a heavy dependency for a narrow requirement.

```
BEGIN:VCALENDAR
VERSION:2.0
PRODID:-//ISRO SAC//CMCMIS Phase 13 Schedule//EN
BEGIN:VEVENT
UID:CAL-2026-04-01@cmcmis.isro.sac
DTSTART;VALUE=DATE:20260401
SUMMARY:CAL: Oscilloscope
END:VEVENT
END:VCALENDAR
```

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 14 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Schedule Routes and Controller Layer

HTTP wiring and permission gates

ROUTE DOCTRINE: Routes define the security boundary: authenticate, authorize, validate, then controller. Controllers stay thin and never own business rules.

Route	Permission	Meaning
GET /schedules	schedule:read-list	List/calendar fetch.
POST /schedules	schedule:create	Create PM/Calibration schedule.
PATCH /schedules/:id	schedule:update	Edit schedule fields.
POST /schedules/:id/status	schedule:update	Lifecycle transition.
DELETE /schedules/:id	schedule:delete	Logical cancel.
GET /schedules/export.ics	schedule:export	Bulk calendar feed.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

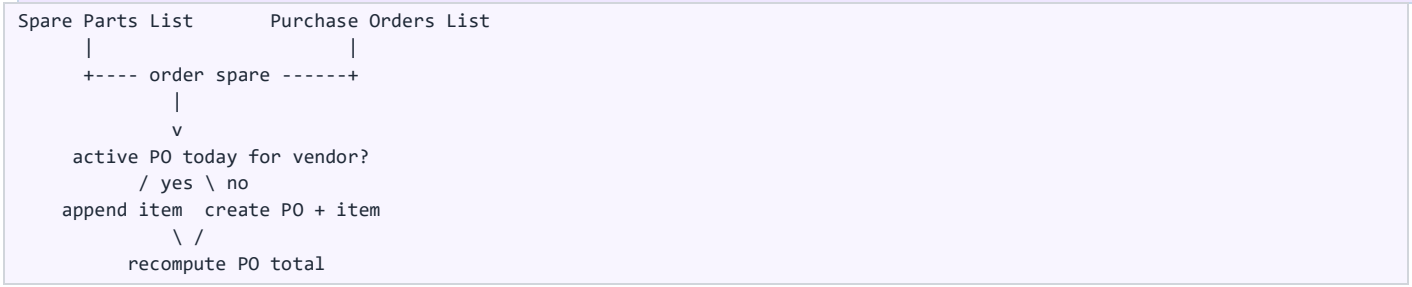
Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 15 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

ARCHITECTURE: Procurement mirrors Schedule: validators define payload contracts, repo owns SQL, service owns transactions and total computation, controller responds, routes enforce RBAC.

FINANCIAL SAFETY: The server computes every line_total and PO total_cost. This avoids trusting user-submitted money totals and creates a single source of truth.

DIAGRAM: Architecture or flow representation below.



How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 16 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Procurement Validators

Payload rules for purchase orders and spares

INPUT SAFETY: Procurement validators protect commercial data: quantity must be positive, unit_cost must be non-negative, warranty months are bounded, and each PO must have at least one item.

Schema	Route	Important validation
poListQuerySchema	GET /purchase-orders	status, vendor, date range, q, pagination.
poCreateSchema	POST /purchase-orders	vendor_id, po_date, items min 1.
poEditSchema	PATCH /purchase-orders/:id	Header/status edit only.
spareCreateSchema	POST /spare-parts	part_name, stock, min_stock, vendor, cost.
spareOrderSchema	POST /spare-parts/:id/order	quantity, unit_cost, warranty, notes.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 17 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Procurement Repository

SQL-only layer for vendor lookup, PO writes and stock rows

SOFT VENDOR FK: Vendor references point to cmms_cont_mst through service validation, not hard database constraints. This preserves legacy flexibility while still preventing invalid input in the app layer.

CODE GENERATORS: nextPoNumber creates PO-YYYY-NNNN. nextSparePartCode creates SP-NNN. Both use FOR UPDATE count logic to reduce duplicate-code race risk.

Function	Purpose	Data risk controlled
findVendorById	Validate vendor soft reference	No invalid vendor rows.
insertPoItem	Insert item + compute line total	Client cannot fake totals.
computePoTotal	Re-sum all item totals	Parent total remains canonical.
findOpenPoForVendor	Reuse today active PO	Avoids unnecessary PO sprawl.
listSpareParts	Stock and low-stock list	Supports procurement decisions.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 18 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Procurement Service

Business rules for PO create/edit and spare ordering

CREATE PO FLOW: The service validates vendor, begins a transaction, generates PO number, inserts header with total zero, inserts item rows, recomputes total from DB rows, writes audit, then commits.

ORDER SPARE FLOW: When ordering a spare, the service finds or creates an active PO for the vendor and day, appends an item, recomputes total, and stamps the spare last_ordered_date.

Method	Main responsibility	Transaction result
createPo	PO header + item rows + total + audit	All-or-nothing PO creation.
editPo	Header/status/note edit	Audit-backed commercial update.
createSpare	New inventory row	Part code + audit.
editSpare	Inventory update	Stock/cost/vendor note updated.
orderSpare	PO reuse/create + line item + spare stamp	Operational order action.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 19 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Frontend Schedule API + Hooks

Clean React Query boundary

API WRAPPERS: src/lib/api/schedule.js unwraps the backend response envelope and handles ICS blob downloads. This keeps page components small and declarative.

HOOK ROOT KEY: useSchedule.js uses a schedules query-key root, so create/edit/transition/cancel can invalidate all schedule views with one call.

Function/hook	Purpose	UI benefit
fetchSchedules	List/calendar data	Both views reuse one endpoint.
downloadScheduleIcsBulk	Export filtered feed	Calendar interoperability.
useSchedules	React Query list hook	Loading/error/fetching state in one place.
useScheduleMutations	Create/edit/transition/cancel	Auto invalidates schedule cache.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 20 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Frontend Schedule Page

Tabs, view toggle, modal entry points

UI STRUCTURE: SchedulePage exposes Preventive Maintenance and Calibration Schedule tabs, plus list/calendar view toggle. The toggle persists in localStorage so the user returns to their preferred layout.

PERMISSION-AWARE UI: Create Schedule appears only with schedule:create. Export .ics appears only with schedule:export. This mirrors backend RBAC rather than relying only on route blocking.

DIAGRAM: Architecture or flow representation below.

```
SchedulePage
  Header: title + Create + Export
  Tabs: Preventive Maintenance | Calibration Schedule
  Toggle: List | Calendar
  Content: ScheduleList or ScheduleCalendar
  Modal: ScheduleFormModal for create/edit
```

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 21 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Frontend Schedule List

DataTable-driven PM and Calibration rows

LIST DESIGN: ScheduleList uses DataTable and dynamically adjusts columns: Calibration shows Priority, PM shows Type. Both show ID, Equipment, Scheduled Date, Status, Engineer and Actions.

STATUS PALETTE: PLANNED, SCHEDULED, DUE, COMPLETED and CANCELLED are displayed with clear badge styling so technical users can scan the operational state fast.

Column	Calibration view	PM view
ID	schedule_code	schedule_code
Equipment	equipment_label	equipment_label
Third column	Priority	Preventive Maintenance type
Status	Badge palette	Badge palette
Actions	Edit + ICS when permitted	Edit + ICS when permitted

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 22 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Frontend Procurement Intent

PO and spare-parts screens planned from backend contract

SOURCE NOTE: The uploaded transcript includes the Procurement backend and route contract. A production frontend page should follow the same pattern as Schedule: api wrapper, React Query hook, tabbed page, create modal, detail drawer, CSV export buttons.

UX DOCTRINE: The procurement UI should not feel like a spreadsheet dump. It should make low-stock risk, vendor linkage, PO totals and order actions visible with minimal clicks.

Planned FE block	Backend endpoint it consumes	Purpose
PO tab	GET /procurement/purchase-orders	View PO headers, status, vendor and totals.
PO detail drawer	GET /purchase-orders/:id	Show items_list and header metadata.
New PO modal	POST /purchase-orders	Create header and line items.
Spare Parts tab	GET /spare-parts	Show inventory, low-stock flag and vendor.
Order action	POST /spare-parts/:id/order	Create/append PO item from inventory row.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 23 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Phase 12 Integration Thinking

Notifications and token messages around Phase 13 mutations

TOKEN BEHAVIOR: Because Phase 12 installed a universal Axios mutation interceptor, successful POST/PATCH/DELETE calls in Schedule and Procurement naturally produce client-side capsule feedback where the URL patterns are covered or by catch-all behavior.

NOTIFICATION CAUTION: The transcript shows Schedule create using an existing notification template as a practical placeholder. For a future hardening slice, dedicated event types like SCHEDULE_CREATED, SCHEDULE_UPDATED, PO_CREATED and SPARE_ORDERED should be added for cleaner user-facing copy.

Event area	Current integration idea	Future cleanup
Schedule create	Notify assigned engineer when present	Add SCHEDULE_CREATED template.
Schedule transition	Audit/history already present	Add SCHEDULE_DUE/COMPLETED notifications.
PO create/order spare	Audit present; tokens via Axios	Add PROCUREMENT_ORDERED notification.
Frontend mutation	Token capsule appears from interceptor	Add explicit message rules for new URLs.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 24 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Security and Row Safety

RBAC, validation, SQL boundaries and audit

SECURITY LAYERS: Phase 13 security is layered: frontend hides buttons, routes enforce permissions, validators reject malformed data, service validates soft references, repo parameterizes SQL, and audit_log records mutations.

NO BLIND TRUST: The system does not trust frontend totals, frontend dates beyond validation, unvalidated equipment references, or vendor strings. Service/repo checks convert user intent into safe database writes.

Layer	Control	Failure prevented
Frontend	Permission-aware buttons	Casual unauthorized attempts.
Routes	authorize(permission)	Direct API access without permission.
Validators	Zod strict schemas	Malformed or extra payload fields.
Service	Soft-FK validation + transactions	Orphan rows and partial writes.
Repo	Parameterized SQL	SQL injection and alias drift.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 25 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Transaction Doctrine

All-or-nothing writes across new tables

SCHEDULE TRANSACTION: Create schedule writes schedules, schedule_status_history and audit_log in one transaction. Status transition updates schedule status, writes history and audit in one transaction.

PROCUREMENT TRANSACTION: Create PO writes header and items, recomputes total, writes audit and commits together. Spare order can create/reuse PO, insert item, recompute total and stamp last_ordered_date in one transaction.

DIAGRAM: Architecture or flow representation below.

```
beginTransaction()
  validate semantic references
  insert/update master row
  insert child rows / history rows
  compute totals or status history
  write audit_log
commit()

if any step fails -> rollback()
```

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 26 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Verification Plan

Smoke checks required for final production seal

HONEST STATUS: The uploaded transcript contains implementation work and file creation. It does not show a final Phase 13 smoke-matrix result in the visible content. Therefore this page defines the verification checklist rather than claiming green verification.

MINIMUM SMOKE MATRIX: The phase should be sealed after migrations apply, endpoints return expected status codes, write flows commit, invalid payloads fail, View-Only cannot mutate, FE builds cleanly, and export files download correctly.

Check	Expected result	Why
Migration 500/501/510	Apply idempotently	Schema and RBAC ready.
GET /schedules	200 with read-list	Schedule list works.
POST /schedules	201 with create permission	Create flow works.
POST /schedules/:id/status	200 or valid conflict	State machine enforced.
POST /spare-parts/:id/order	201 and PO total updated	Procurement order flow works.
View-Only write attempts	403	RBAC is correct.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 27 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

Risk Register

Senior engineer review before final deployment

RISK THINKING: The phase is architecturally aligned, but production handover should still review race conditions, money precision, notification templates, and frontend completeness.

Risk	Impact	Mitigation
FOR UPDATE count generator under high concurrency	Duplicate code risk if isolation is weak	Consider dedicated sequence table in hardening.
Soft FK references	Legacy row rename may stale labels	Service validation + denormalized labels; refresh on edit.
ICS line folding edge cases	Some calendar clients may reject long text	foldLine exists; test Outlook/GCal/Apple.
Procurement monetary precision	Rounding drift	Store DECIMAL, recompute total from line rows.
Notification copy reuse	Generic messages	Add Phase 13-specific templates in next patch.

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 28 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

HANDOVER RULE: Do not deploy Phase 13 by copying frontend only or backend only. Schema, RBAC, backend routes, frontend routes, token feedback, and smoke tests must move together.

Area	Checklist item	Owner
Database	Run migrations 500, 501, 510 and verify tables/indexes/permissions	Backend/DB
Backend	Mount schedules and procurement routes in server.js	Backend
Frontend	Wire /schedule and /procurement protected routes	Frontend
RBAC	Check sidebar visibility for all five roles	Full-stack
Exports	Test .ics and CSV downloads in browser	QA
Audit	Confirm audit_log rows for create/update/order	Backend/QA

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 29 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.

LOCKED SUMMARY: Phase 13 introduces the Schedule and Procurement operational layer: new additive tables, dedicated permissions, backend modules, transaction-safe services, calendar export, CSV export, and frontend Schedule list foundation.

NEXT ENGINEERING MOVE: Finish any remaining frontend Procurement screens, add Phase 13-specific notification templates, run the full smoke matrix, then seal the phase with a final green verification document.

DS NOTE: This document is intentionally limited to thirty rendered pages, with page-by-page writing and manual page breaks to match the requested style.

Final state	Meaning
Document length	30 pages only
Filename	phase13DSstyleSEALED30Pages.docx
Style	DS-style structured blocks, tables, diagrams, definitions and light code
Source	Uploaded Phase 13 pasted transcript

How to think for logic development:

Read this page as a contract boundary. First identify which layer owns the decision, then verify the next layer receives only clean data. For backend work, follow the chain: route permission -> validator contract -> service semantic check -> repository SQL -> transaction commit -> audit/notification/cache effect. This prevents UI-first debugging and keeps the phase senior-engineered.

DS IMPLEMENTATION NOTE: The page is intentionally written before switching to the next page. Keep this method when writing code too: finish one layer, verify it, then move to the next layer. That is how CMCMIS stays locked instead of becoming scattered.

Senior expansion:

For this phase, the most important engineering habit is separation of responsibility. Database migrations should express durable structure only. Backend repositories should express SQL only. Services should express business decisions and transactions only. Controllers should not become mini-services. Frontend pages should render state and call hooks, not rebuild backend rules. When each file owns only one responsibility, the system remains readable for the next engineer, auditable for government-grade review, and easier to debug under pressure.

Review focus: verify that the concept on this page has a clear database contract, API contract, role permission, audit path, and UI behavior. If any one of these five is missing, the feature is not yet production sealed.

DS page note: Page 30 is part of the fixed 30-page Phase 13 package. It is written as a standalone explanation unit before moving to the next page.